

SESSION 1: MULTI-SCALE PARALLELISM STRATEGIES

Data Parallelism Fundamentals

Distributing Datasets for Parallel Training

Core Concept:

Data parallelism splits training datasets across multiple devices, with each device maintaining a complete copy of the model. This approach scales naturally with dataset size and provides linear speedup under ideal conditions.

Mathematical Foundation:

For N devices processing batch size B :

- Global gradient equals to the average of local gradients: $g_{global} = (1/N) \times \sum(g_i)$ where g_i is local gradient
- Equivalent to single device with batch size $N \times B$
- Maintains SGD (Stochastic Gradient Descent) mathematical properties

Device Coordination:

1. Dataset splitting into non-overlapping subsets
2. Independent forward and backward passes
3. Gradient synchronization via ring all-reduce
4. Synchronized parameter updates

Scaling Benefits:

Linear speedup possible up to communication bottleneck threshold.

Quick Review – Stochastic Gradient Descent

The core idea of SGD is to **update the model's parameters** using the **gradient** calculated from a **small, randomly selected subset of the training data**, known as a mini-batch, rather than the entire dataset. In its purest form, SGD updates parameters using only a single data point at a time, though using mini-batches is more common in practice and often referred to as SGD.

- **Mathematical Formulation:** For model parameters ϑ , learning rate η , and loss $L(\vartheta)$:
- **Standard Gradient Descent:** $\vartheta_{t+1} = \vartheta_t - \eta \nabla L(\vartheta_t)$ (uses full dataset)
- **Stochastic Gradient Descent:** $\vartheta_{t+1} = \vartheta_t - \eta \nabla L_i(\vartheta_t)$ (uses single random example i)
- **Mini-batch SGD (most common):** $\vartheta_{t+1} = \vartheta_t - \eta \nabla LB(\vartheta_t)$ - Where B is a mini-batch of examples and $\nabla LB(\vartheta_t) = (1/|B|) \sum_{i \in B} \nabla L_i(\vartheta_t)$

Algorithm Steps

1. Initialize model parameters ϑ_0 randomly
2. Shuffle the training dataset
3. For each epoch (pass through the dataset):
 - For each mini-batch B in dataset:
 - Compute gradient $\nabla LB(\vartheta_t)$
 - Update parameters: $\vartheta_{t+1} = \vartheta_t - \eta \nabla LB(\vartheta_t)$
4. Repeat for multiple epochs until the loss becomes sufficiently small or stops improving

Example Update Rule: For linear regression with MSE loss: $L(\vartheta) = (1/2n) \sum (y_i - \vartheta^T x_i)^2$

SGD update for parameter ϑ_j : $\vartheta_j \leftarrow \vartheta_j + \eta (y_i - \vartheta^T x_i) x_{ij}$

Data Parallelism Mechanics

Data Parallelism Fundamentals

Step-by-Step Execution Process

Execution Flow:

Input Data → Split into Batches → Distribute to GPUs



GPU 1: Forward Pass → Backward Pass → Compute Gradients

GPU 2: Forward Pass → Backward Pass → Compute Gradients

GPU 3: Forward Pass → Backward Pass → Compute Gradients

GPU 4: Forward Pass → Backward Pass → Compute Gradients



Gradient Synchronization (Ring All-Reduce)



Parameter Update on All Devices → Next Iteration

Communication Pattern:

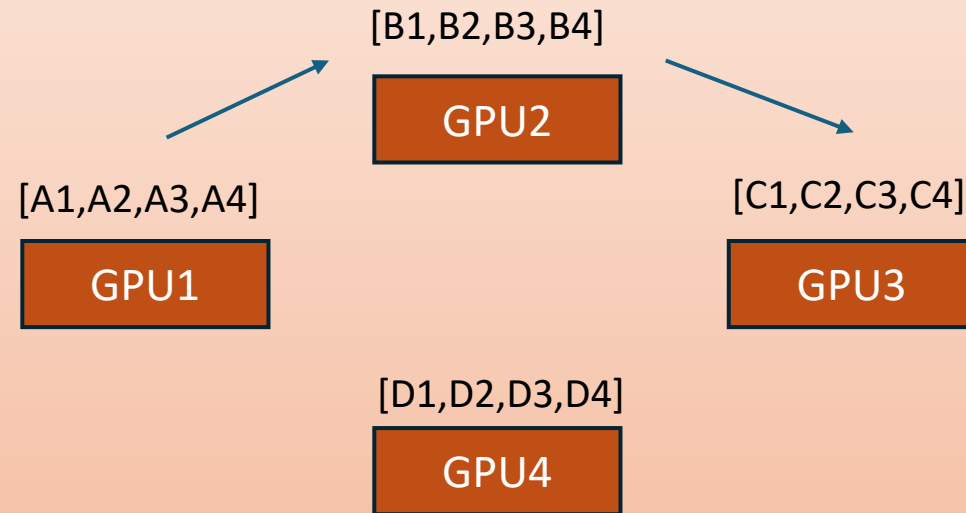
Ring all-reduce algorithm minimizes communication overhead:

- Each device communicates only with neighbors
- 7 steps for 8 devices vs. 56 for naive peer-to-peer
- Bandwidth usage scales optimally with device count

Implementation Example:

Modern frameworks like PyTorch DistributedDataParallel handle synchronization automatically.

Ring All-Reduce example



1. Initial State: Each GPU splits its gradient into 4 chunks (1 per GPU).

SESSION 1: CLASSICAL DISTRIBUTED TRAINING

*In data parallelism, the **batch size** is the total number of training examples processed together before the model is updated.*

*For example, if you have a total batch size of 64 and 4 devices, each device processes a slice of 16 examples. These individual slices are often called **minibatches**.*

Distributing Training Data Across Devices

Mathematical Foundation:

- In standard SGD with batch size B , the gradient update is:

$$g = \frac{1}{|B|} \sum_{xi \in B} \nabla_{\theta} L(\theta, xi)$$

- With N devices, rank k computes:

$$g_k = \frac{1}{|B_k|} \sum_{xi \in B_k} \nabla_{\theta} L(\theta, xi)$$

- The global gradient is

$$g_{global} = \sum_{k=1}^N \frac{|B_k|}{B} g_k, \quad B = \sum_{k=1}^N |B_k|$$

- For equal local batch sizes:

$$g_{global} = \frac{1}{N} \sum_{k=1}^N g_k$$

Process Overview:

- Split the dataset into non-overlapping subsets
- Perform independent forward/backward passes
- Aggregate gradients using all-reduce
- Apply the same optimizer update on every device

Data Parallelism Implementation

DDP stands for Distributed Data Parallel, a training wrapper that replicates a model on multiple devices and averages gradients across them via collectives like all-reduce. After synchronization, each replica applies the same optimizer step so all copies stay in lockstep for correct, scalable training.

Steps

1. Deterministic, non-overlapping data sharding per rank
2. Forward + backward per device independently
3. Collective gradient synchronization (all-reduce)
4. Identical parameter updates on all devices

Good practices

- Use a seeded `DistributedSampler` for dataloaders
- Overlap communication and compute via gradient bucketing
- Configure DDP for unused params when models branch (`find_unused_parameters`)
- Gradient clipping before the optimizer step
- Mixed precision: prefer BF16 if available; otherwise FP16 with dynamic loss scaling

Notes on synchronization

- Ring all-reduce minimizes bandwidth and scales with $N-1$ hops
- Ensure optimizer steps are identical across ranks after sync

Communication Optimization

BF16 is a 16-bit floating-point format that sacrifices precision for a wide dynamic range, making it stable for training neural networks. FP16 is a 16-bit format that offers higher precision than BF16 but has a narrower range, which can cause overflow or underflow issues during training.

ResNet-50 is a popular model for image recognition whose key innovation was a "shortcut" that allows the core signal, or meaningful information, to skip over layers. These shortcuts prevent the signal from getting weak and lost in deep networks, providing a direct path that makes them much easier to train accurately.

How big are the gradients?

- Gradient size = P params $\times$ b bytes/param
- FP32 = 4 bytes; BF16/FP16 = 2 bytes

Ring all-reduce traffic per device:

- $2 \times P \times b \times (N-1)/N$

Example (ResNet-50 $\approx$ 25M params)

- FP32 gradients $\approx$ 100 MB
- $N = 8$ GPUs $\rightarrow \approx$ 175 MB traffic per device per step

Fabrics matter

- NVLink $\sim$ 300 GB/s $\rightarrow \sim$ 0.6 ms for 175 MB (idealized)
- 100 Gbps Ethernet/InfiniBand $\approx$ 12.5 GB/s $\rightarrow \sim$ 14 ms (idealized)

Make it faster

- Overlap comm with compute via gradient bucketing
- Choose bucket sizes to avoid many tiny messages
- Use BF16/FP16 gradients to halve traffic (with proper scaling)
- For small tensors, tree-based reductions can beat ring due to lower latency

Scalability Analysis

Sources of scaling loss

- Communication overhead: bandwidth limits, latency, protocol inefficiency
- Synchronization barriers and stragglers: everyone waits for the slowest
- Load imbalance/heterogeneity: uneven work, throttling, ECC, I/O skew
- Memory pressure: activations/optimizer state constrain batch/parallelism

Measuring efficiency

- Ideal speedup = N
- Actual speedup = T_1 / T_N
- Parallel efficiency = $(T_1 / T_N) / N$

Amdahl's Law (upper bound on speedup)

- Speedup $\leq 1 / (s + (1 - s)/N)$, where s = serial fraction
- Example: $s = 0.10 \Rightarrow \text{max speedup} = 10\times \text{ as } N \rightarrow \infty$
- Even small s severely caps efficiency at large N

What to do (reduce s , hide comm)

- Overlap communication with compute: gradient bucketing, fused kernels
- Hierarchical collectives; topology-aware placement
- Lower-precision or compressed gradients with error feedback
- Algorithmic options: increase local batch; partial/delayed sync; asynchronous variants with care
- Balance work: uniform microbatching, stage rebalancing, dynamic batch assignment
- Detect and mitigate stragglers: health checks, retry, prefetch, pinned workers

Distributed Systems Concerns in Practice

Fabrics and topology:

- Intra-node: PCIe, NVLink
- Inter-node: 25/100/200/400 Gbps Ethernet/InfiniBand; latencies 2–10× higher

Collectives vs parameter server:

- Collectives (NCCL/Gloo/MPI): bandwidth-efficient, symmetric
- PS: flexible, supports async, but can bottleneck at servers

Orchestration & launch:

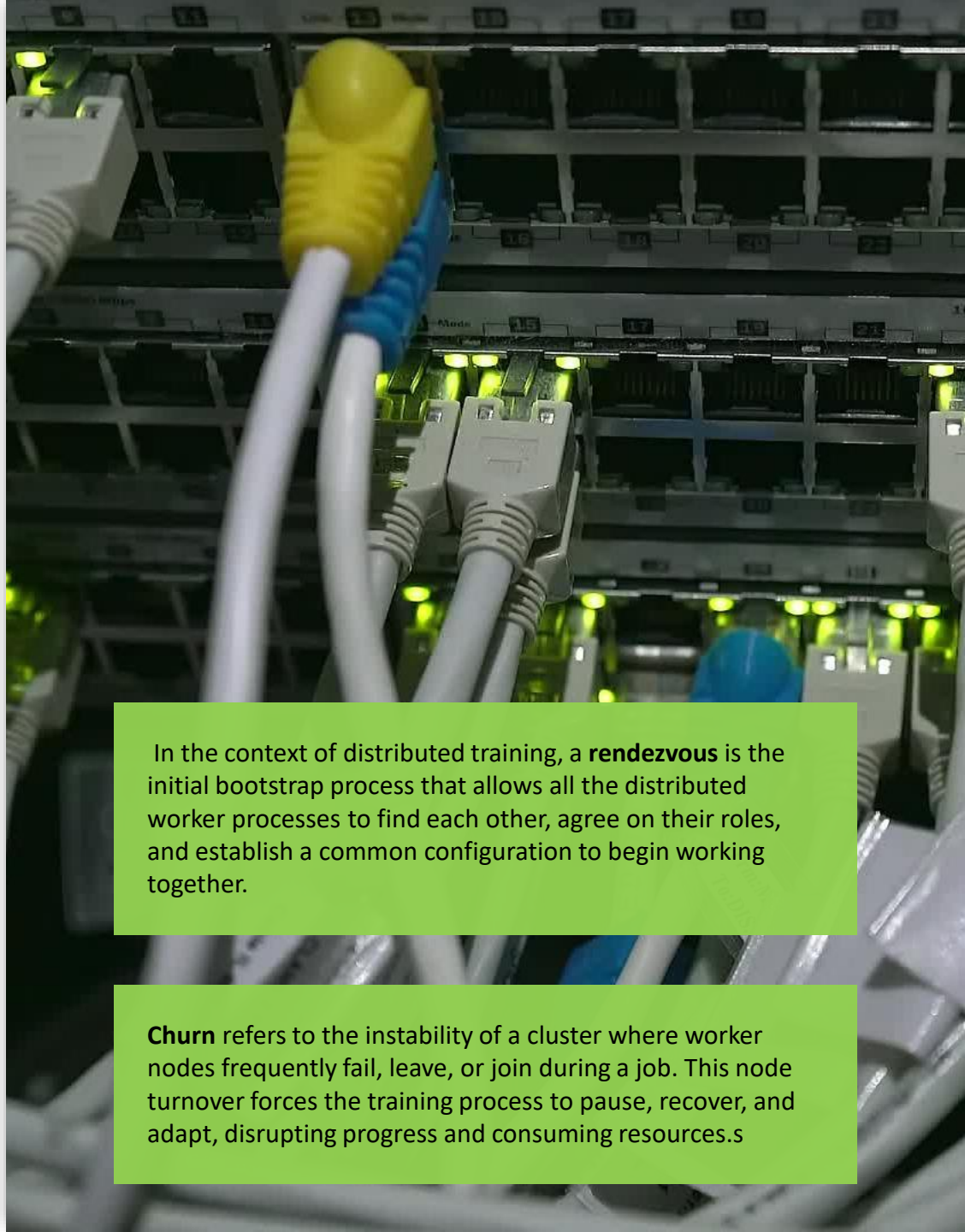
- SLURM/Kubernetes; ranks, world size, rendezvous; topology-aware placement

Reproducibility:

- Deterministic sharding and seeding; avoid nondeterministic kernels

Elasticity:

- Handle node churn; reshard data and adjust LR safely



In the context of distributed training, a **rendezvous** is the initial bootstrap process that allows all the distributed worker processes to find each other, agree on their roles, and establish a common configuration to begin working together.

Churn refers to the instability of a cluster where worker nodes frequently fail, leave, or join during a job. This node turnover forces the training process to pause, recover, and adapt, disrupting progress and consuming resources.

Sync vs Async vs Elastic Data Parallel

Synchronous all-reduce:

- Pros: stable convergence; single-node equivalence
- Cons: straggler-sensitive; per-step barrier

Parameter-server (async/stale-sync):

- Pros: more throughput; tolerant to slow clients
- Cons: stale gradients; careful LR tuning; potential accuracy hit

Elastic DP (synchronous with changing world size):

- World size adjusts at runtime; reshard data/checkpoints; adjust LR with effective batch

Guidance: prefer sync for accuracy-critical; use async/elastic when churn is inevitable